Classes and Object Lifetime: Mock Test

Classes and object lifetime, 50 marks

COS 110

Swift Tuition | 2026

Before you begin

Write this paper in one sitting before you open the solutions. Section A has short questions (22 marks). Section B uses one scenario, the Greenhouse classes (28 marks); questions 16 to 18 are drawn in DrawIO.

Course information	COS 110 support
Solutions	the solutions page of 2. Classes and Object Lifetime

Total: 50 marks. **Suggested time:** 90 minutes (**Assumed**: Semester Test 1 was about 50 marks; its time is not recorded).

C++98 only (`g++ -std=c++98 -Wall -Werror`). Write member functions outside the class unless a question says otherwise. Questions 16 to 18 are drawn in DrawIO; in a test they are uploaded as files.

Questions

1. Default access in a struct and a class	1	p. 4
2. Empty brackets after an object name	1	p. 4
3. Copy constructor or assignment operator	1	p. 4
4. One constructor, two jobs	2	p. 5
5. Trace a static counter	3	p. 5
6. Trace constructors, copies and destructors	4	p. 6
7. Find the errors: const, static and a const member	4	p. 7
8. Three meanings of const	3	p. 8
9. The copy constructor parameter	2	p. 8
10. Many constructors, one destructor	1	p. 9
11. Visibility to C++	1	p. 10
12. Define the static member	1	p. 10
13. Does it need a destructor?	2	p. 11
14. Implement the Sensor constructor and destructor	3	p. 11
15. Implement the Greenhouse constructor	4	p. 11
16. Flowchart: the Greenhouse assignment operator	7	p. 13
17. Object diagram after line 8	5	p. 14
18. Code to class diagram	5	p. 15

Section A: short questions (22 marks)

Question 1: Default access in a struct and a class [1]

Which marked line does not compile?

main.cpp

```cpp
#include <iostream>

struct Record {
  int value;
};

class Counter {
  int count;

 public:
  void set(int c) {
    count = c;
  }
};

int main() {
  Record r;
  Counter k;
  r.value = 5;   // line A
  k.set(5);      // line B
  k.count = 5;   // line C
  std::cout << r.value << std::endl;
  return 0;
}
```

A. Line A only

B. Line B only

C. Line C only

D. Lines B and C

Question 2: Empty brackets after an object name [1]

The class `Author` has the constructor `Author(std::string n = "Unknown", int b = 0)`. What does the statement `Author a3();` inside `main` do?

A. It creates an Author called a3 with the default values ”Unknown” and 0.

B. It declares a function a3 that takes no parameters and returns an Author.

C. It creates a3 on the heap.

D. It calls the copy constructor.

Question 3: Copy constructor or assignment operator [1]

`Box` has a copy constructor and an assignment operator. `b1` and `b2` already exist. Which statement calls the copy constructor?

A. b2 = b1;

B. Box b3 = b1;

C. byReference(b1); where the parameter is const Box&

D. Box b4(4);

Question 4: One constructor, two jobs [2]

Complete the sentence about this constructor:

```
Item(float value = 0.0f, int size = 0, std::string text = "");
```

(a) It is a _____ constructor because ...

(b) It is also a _____ constructor because ...

Question 5: Trace a static counter [3]

Write down exactly what this program prints.

main.cpp

```
#include <iostream>

class Ticket {
 private:
  int number;
  static int issued;
  static int active;

 public:
  Ticket() {
    issued++;
    active++;
    number = issued;
  }
  ~Ticket() {
    active--;
  }
  void print() const {
    std::cout << number << "/" << active << " ";
  }
  static int getIssued() {
    return issued;
  }
```

```
};

int Ticket::issued = 0;
int Ticket::active = 0;

int main() {
  Ticket a;
  a.print();
  for (int i = 0; i < 2; i++) {
    Ticket t;
    t.print();
  }
  Ticket* p = new Ticket;
  p->print();
  delete p;
  Ticket b;
  b.print();
  std::cout << Ticket::getIssued() << std::endl;
  return 0;
}
```

Question 6: Trace constructors, copies and destructors [4]

Write down exactly what this program prints. It prints two lines.

main.cpp

```
#include <iostream>
#include <string>

class Note {
 private:
  std::string text;

 public:
  Note(std::string t = "blank") : text(t) {
    std::cout << "N(" << text << ") ";
  }
  Note(const Note& o) : text(o.text + "*") {
    std::cout << "C(" << text << ") ";
  }
  Note& operator=(const Note& o) {
    text = o.text;
    std::cout << "A(" << text << ") ";
    return *this;
  }
  ~Note() {
    std::cout << "~" << text << " ";
  }
};

class Folder {
```

```
 private:
  Note label;

 public:
  Folder(std::string t) : label(t) {
    std::cout << "F ";
  }
  ~Folder() {
    std::cout << "~F ";
  }
};

void show(Note n) {
  std::cout << "show ";
}

int main() {
  Note a("x");
  Note b = a;
  show(b);
  {
    Folder f("y");
  }
  Note c;
  c = a;
  std::cout << std::endl;
  return 0;
}
```

Question 7: Find the errors: const, static and a const member [4]

The lines marked L1 to L5 start a definition. For each marked line, say whether it has a compilation error. If it does, explain the error in one sentence. Marks are subtracted if correct code is called an error. Then rewrite the definition that starts at L2 so that it compiles. (3 marks for the errors with reasons, 1 mark for the rewrite.)

main.cpp

```
#include <iostream>

class Meter {
 private:
  double value;
  static int meterCount;
  const int serial;

 public:
  Meter(double v, int s);
  double getValue() const;
  void reset() const;
  static int getCount();
```

```
};

int Meter::meterCount = 0;                          // L1

Meter::Meter(double v, int s) {                     // L2
  value = v;
  serial = s;
  meterCount++;
}

double Meter::getValue() const {                    // L3
  return value;
}

void Meter::reset() const {                         // L4
  value = 0;
}

static int Meter::getCount() {                      // L5
  return meterCount;
}

int main() {
  Meter m(2.5, 7);
  std::cout << m.getValue() << " " << Meter::getCount() << std::endl;
  return 0;
}
```

Question 8: Three meanings of const [3]

The class `Plot` declares this member function:

```
const Plot& larger(const Plot& other) const;
```

For each of the three `const` keywords, write one statement that the keyword would stop from compiling, and say where the statement would be (inside `larger` or in the calling code).

Question 9: The copy constructor parameter [2]

A copy constructor is declared as `Greenhouse(const Greenhouse& other);`.

(a) Why must the parameter be a reference and not `Greenhouse other`? (1)

(b) Why is the reference `const`? (1)

Question 10: Many constructors, one destructor [1]

A class can have several constructors but only one destructor. Explain why.

Section B: the Greenhouse scenario (28 marks)

Question 11: Visibility to C++ [1]

Questions 11 to 17 use the Greenhouse class diagram in Figure 1. A greenhouse owns its beds: when it is destroyed, its beds are destroyed too. It only points at sensors, which are created and destroyed outside it, and several greenhouses may share one sensor. A gardener only uses a greenhouse while inspecting it.

Figure 1: Greenhouse class diagram

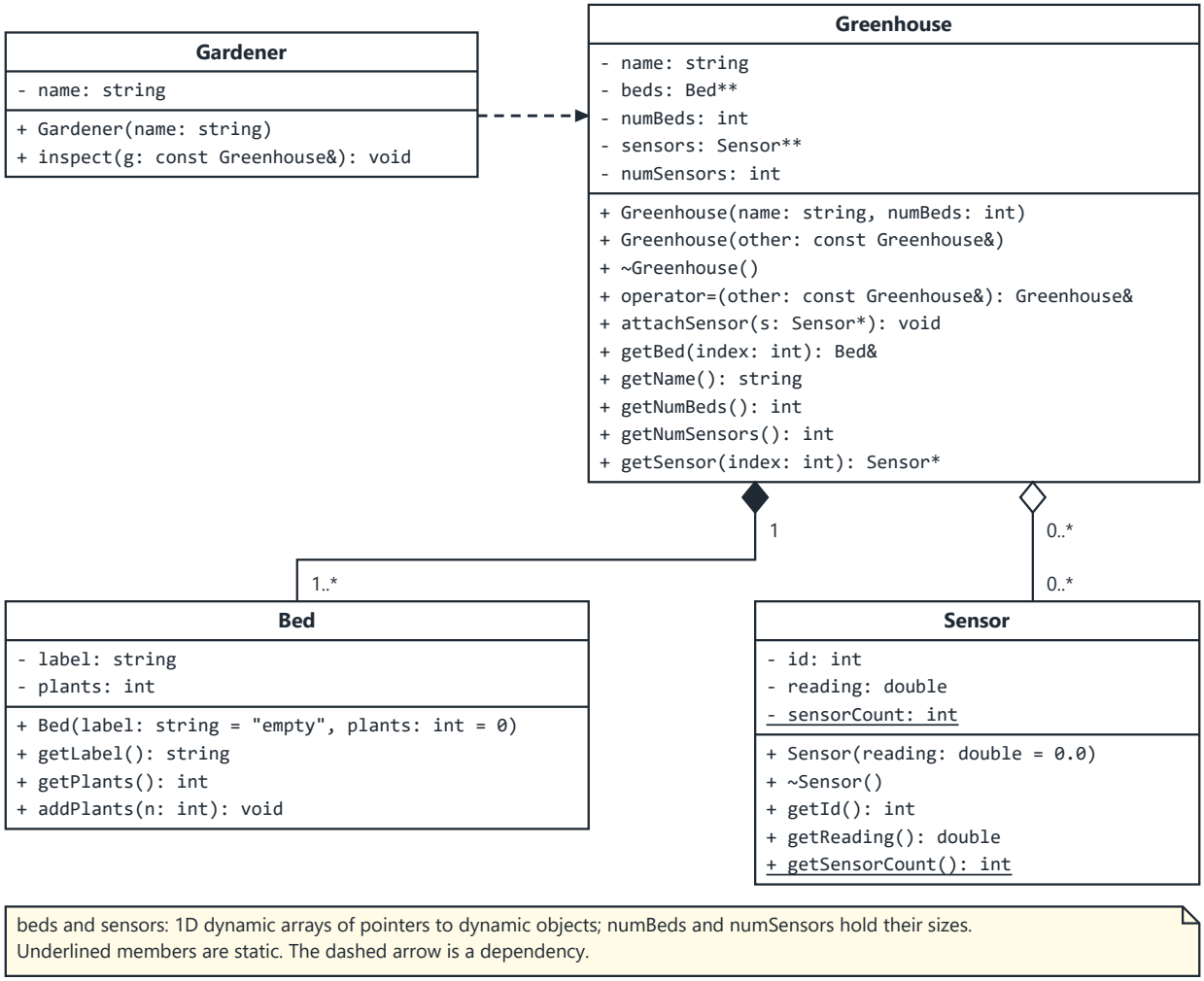


Figure 1: Greenhouse class diagram

C++98 only. Write member functions outside the class (no inline code).

(a) Under which C++ access specifier is `beds` declared in `Greenhouse.h`? (0.5)

(b) Under which C++ access specifier is `attachSensor` declared? (0.5)

Question 12: Define the static member [1]

In Figure 1, `sensorCount` is underlined. Write the single C++ statement in `Sensor.cpp` that defines it and sets it to 0.

Question 13: Does it need a destructor? [2]

(a) Does `Bed` need a destructor that you write yourself? Explain. (1)

(b) Does `Greenhouse` need one? Explain, using the relationships in Figure 1. (1)

Question 14: Implement the Sensor constructor and destructor [3]

Implement the `Sensor` constructor and destructor outside the class (in `Sensor.cpp`). (2 marks for the constructor, 1 for the destructor.)

- The constructor must use an initialisation list for `reading`, or it earns no marks.
- Each new sensor adds 1 to `sensorCount`, and its `id` is the new value of `sensorCount` (the first sensor gets id 1).
- The destructor takes 1 off `sensorCount`.

Question 15: Implement the Greenhouse constructor [4]

Implement `Greenhouse(name: string, numBeds: int)` outside the class. (4 marks: one for each point below.)

- Use an initialisation list for `name` and `numBeds`.
- If `numBeds` is less than 1, throw the string ”Invalid number of beds” before any memory is allocated.
- Otherwise create the `beds` array with one default `Bed` object in each element.

- A new greenhouse has no sensors: `sensors` is NULL and `numSensors` is 0.

Question 16: Flowchart: the Greenhouse assignment operator [7]

Draw a flowchart, in DrawIO, for `operator=(other: const Greenhouse&): Greenhouse&`. Use COS 132 notation and describe each step in words; nodes that only hold code lose marks.

- The object must end with the same name, the same beds (as its own copies) and the same sensors as `other`.
- No memory may leak, and nothing may be deleted twice. Use the relationships in Figure 1 to decide what is copied and what is shared.
- Assigning an object to itself must leave it unchanged.

Save the file as `Q16.drawio` (**Assumed** file name).

Question 17: Object diagram after line 8 [5]

The program below uses the Greenhouse classes of Figure 1. In DrawIO, draw the UML object diagram of every object that is alive straight after line 8 has run. Show each object's attribute values, the array elements as objects (for example `beds[0]:Bed`), and the links between objects. Show the value of the static `sensorCount` in a note. Save the file as `Q17.drawio` (**Assumed** file name).

`main.cpp`

```
#include <iostream>
#include "Greenhouse.h"

int main() {
  Sensor s1(21.5);                    // line 1
  Sensor* s2 = new Sensor(19.0);      // line 2
  Greenhouse north("North", 2);       // line 3
  north.getBed(0).addPlants(12);      // line 4
  north.attachSensor(&s1);            // line 5
  north.attachSensor(s2);             // line 6
  Greenhouse south(north);            // line 7
  south.getBed(1).addPlants(5);       // line 8
  delete s2;                          // line 9
  std::cout << Sensor::getSensorCount() << std::endl;
  return 0;
}
```

Question 18: Code to class diagram [5]

In DrawIO, draw the complete UML class diagram for the classes in this program: every attribute and operation with its visibility and types, static members underlined, the relationships between the classes with their multiplicities, and a note for each dynamic array. Use the comments above `Clinic` to decide the relationships. Save the file as `Q18.drawio` (**Assumed** file name).

`main.cpp`

```cpp
#include <iostream>
#include <string>

class Doctor {
 private:
  std::string name;

 public:
  Doctor(std::string name) : name(name) {}
  std::string getName() const {
    return name;
  }
};

class Patient {
 private:
  std::string name;
  int age;

 public:
  Patient(std::string name, int age) : name(name), age(age) {}
  int getAge() const {
    return age;
  }
};

class Calendar {
 public:
  bool isOpen(int day) const {
    return day >= 1 && day <= 5;
  }
};

// A clinic owns its patients' records: they are deleted with the clinic.
// The doctor on call works at many clinics and is never deleted by a clinic.
class Clinic {
 private:
  std::string name;
  Patient** patients;
  int numPatients;
  Doctor* onCall;
  static int clinicCount;

 public:
  Clinic(std::string name, int numPatients);
  ~Clinic();
  void setOnCall(Doctor* d);
  bool canBook(const Calendar& c, int day) const;
  static int getClinicCount();
};

```

```cpp
int Clinic::clinicCount = 0;

Clinic::Clinic(std::string name, int numPatients)
    : name(name), numPatients(numPatients), onCall(NULL) {
  patients = new Patient*[numPatients];
  for (int i = 0; i < numPatients; i++) {
    patients[i] = new Patient("unknown", 0);
  }
  clinicCount++;
}

Clinic::~Clinic() {
  for (int i = 0; i < numPatients; i++) {
    delete patients[i];
  }
  delete[] patients;
  clinicCount--;
}

void Clinic::setOnCall(Doctor* d) {
  onCall = d;
}

bool Clinic::canBook(const Calendar& c, int day) const {
  return onCall != NULL && c.isOpen(day);
}

int Clinic::getClinicCount() {
  return clinicCount;
}

int main() {
  Doctor dlamini("Dr Dlamini");
  Calendar week;
  Clinic hatfield("Hatfield", 2);
  hatfield.setOnCall(&dlamini);
  std::cout << hatfield.canBook(week, 3) << " " << Clinic::getClinicCount() <<
    std::endl;
  return 0;
}
```